

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
Sync Scalper (M15/M5 Sync) - Pine Script v5
```

```
-----
```

که فرستادی، بدون هیچ قانون اضافه TradingView ترجمه‌ی مستقیم و کامل استراتژی

- LH یا HH/HL ساختار سوینگ + EMA شیب + EMA50 (M15) بایاس روند از تایم‌فریم بالاتر
- AT بر حسب) EMA فاصله از ،EMA50 قیمت نسبت به : (M5) سیگنال ورود روی تایم‌فریم چارت
- RSI کندل تاییدی (نسبت بدنه به رنج)، فیلتر ، (ضد بازار رنج) ADX فیلتر
- (کندل چپ/راست با pivotLen یا pivot) M5 آخرین سوینگ‌پوینت : SL
- (rrMin) ریسک × حداقل نسبت ریسک/ریوارد : TP
- فاصله‌ی ریسک $\div$ equity سایز پوزیشن: درصد ریسک از
- حداکثر تعداد سیگنال در روز
- ت offline چون بکتست) اسلیپیج نادیده گرفته شده ، (Pine Script مطابق) %کارمزد 0.05

اجرا (روی Replit/Colab):

```
pip install ccxt pandas numpy
python sync_scalper_backtest.py
```

```
"""
```

```
import ccxt
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
```

```
# ----- Pine Script) دقیقاً مطابق ورودی‌های) تنظیمات -----
```

```
SYMBOL = "XRP/USDT"
```

```
HTF = "15m"      # Higher Timeframe (M15)
```

```
LTF = "5m"       # تایم‌فریم چارت (M5)
```

```
MONTHS_BACK = 6
```

```
INITIAL_CAPITAL = 10000.0
```

```
COMMISSION_PCT = 0.05 # درصد کارمزد هر طرف معامله
```

```
RISK_PCT = 0.75      # Risk % per trade
```

```
RR_MIN = 2.0         # Minimum Risk:Reward
```

```
MAX_SIGNALS_PER_DAY = 1
```

```
EMA_LEN = 50
```

```
EMA_SLOPE_LOOKBACK = 3
```

```
RSI_LEN = 14
```

```
RSI_BUY_MIN = 45
```

```
RSI_SELL_MAX = 55
```

```

PIVOT_LEN = 5 # Pivot Left/Right bars (سوینگ SL هم M15 هم برای ساختار)

ADX_LEN = 14
ADX_MIN = 18
MAX_ATR_DIST = 2.5 # کمتر "خیلی دور نشده" - (ATR ضریب) EMA50 حداکثر فاصله از
PULLBACK_ATR_MAX = 1.0 # (ATR ضریب) EMA50 حداکثر فاصله‌ی پولبک تا

MIN_BODY_RATIO = 0.4 # حداقل نسبت بدنه‌ی کندل به رنج کل کندل

def fetch_ohlcv(exchange, symbol, timeframe, months_back):
    since = exchange.parse8601(
        (datetime.utcnow() - timedelta(days=30 * months_back)).strftime("%Y-%m-%d")
    )
    all_candles = []
    limit = 1500
    while True:
        candles = exchange.fetch_ohlcv(symbol, timeframe=timeframe, since=since,
        if not candles:
            break
        all_candles += candles
        since = candles[-1][0] + 1
        if len(candles) < limit:
            break
    df = pd.DataFrame(all_candles, columns=["timestamp", "open", "high", "low", "
    df["timestamp"] = pd.to_datetime(df["timestamp"], unit="ms")
    return df

# ----- TradingView همه با هموارسازی وایلدِر، مطابق) اندیکاتورها -----

def ema(series, length):
    return series.ewm(span=length, adjust=False).mean()

def rsi(series, length):
    delta = series.diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)
    avg_gain = gain.ewm(alpha=1 / length, adjust=False).mean()
    avg_loss = loss.ewm(alpha=1 / length, adjust=False).mean()
    rs = avg_gain / avg_loss.replace(0, np.nan)
    out = 100 - (100 / (1 + rs))
    return out.where(avg_loss != 0, 100.0)

```

```

def atr(df, length):
    prev_close = df["close"].shift(1)
    tr = pd.concat([
        df["high"] - df["low"],
        (df["high"] - prev_close).abs(),
        (df["low"] - prev_close).abs(),
    ], axis=1).max(axis=1)
    return tr.ewm(alpha=1 / length, adjust=False).mean()

def dmi_adx(df, length):
    """Pine Script در ta.dmi() دقیقاً مطابق - (+DI, -DI, ADX) خروجی: """
    high, low, close = df["high"], df["low"], df["close"]
    prev_high, prev_low, prev_close = high.shift(1), low.shift(1), close.shift(1)

    tr = pd.concat([
        high - low, (high - prev_close).abs(), (low - prev_close).abs(),
    ], axis=1).max(axis=1)

    up_move = high - prev_high
    down_move = prev_low - low
    plus_dm = pd.Series(0.0, index=df.index)
    minus_dm = pd.Series(0.0, index=df.index)
    plus_mask = (up_move > down_move) & (up_move > 0)
    minus_mask = (down_move > up_move) & (down_move > 0)
    plus_dm[plus_mask] = up_move[plus_mask]
    minus_dm[minus_mask] = down_move[minus_mask]

    atr_w = tr.ewm(alpha=1 / length, adjust=False).mean().replace(0, np.nan)
    plus_di = 100 * (plus_dm.ewm(alpha=1 / length, adjust=False).mean() / atr_w)
    minus_di = 100 * (minus_dm.ewm(alpha=1 / length, adjust=False).mean() / atr_w)
    di_sum = (plus_di + minus_di).replace(0, np.nan)
    dx = 100 * (plus_di - minus_di).abs() / di_sum
    adx_val = dx.ewm(alpha=1 / length, adjust=False).mean()
    return plus_di, minus_di, adx_val

def pivot_points(df, left, right):
    """
    `left` آن نسبت به high/low پیوت است اگر i کندل ta.pivohigh/pivotlow معادل
    (non-repainting) کندل بعد بیشترین/کمترین باشد. غیرقابل تکرار `right` کندل قبل و
    به بعد "دیده" میشود i+right فقط از کندل i پیوت کندل
    """
    n = len(df)
    is_pivot_high = pd.Series(False, index=df.index)
    is_pivot_low = pd.Series(False, index=df.index)
    highs, lows = df["high"].values, df["low"].values

```

```

for i in range(left, n - right):
    window_h = highs[i - left: i + right + 1]
    window_l = lows[i - left: i + right + 1]
    if highs[i] == window_h.max() and (highs[i] > highs[i - left:i]).all() \
        and (highs[i] > highs[i + 1:i + right + 1]).all():
        is_pivot_high.iloc[i] = True
    if lows[i] == window_l.min() and (lows[i] < lows[i - left:i]).all() \
        and (lows[i] < lows[i + 1:i + right + 1]).all():
        is_pivot_low.iloc[i] = True
return is_pivot_high, is_pivot_low

def add_htf_indicators(df):
    df["ema"] = ema(df["close"], EMA_LEN)
    df["pivot_high"], df["pivot_low"] = pivot_points(df, PIVOT_LEN, PIVOT_LEN)
    return df

def add_ltf_indicators(df):
    df["ema50"] = ema(df["close"], EMA_LEN)
    df["rsi14"] = rsi(df["close"], RSI_LEN)
    df["atr5"] = atr(df, 14)
    _, _, df["adx"] = dmi_adx(df, ADX_LEN)
    df["pivot_high"], df["pivot_low"] = pivot_points(df, PIVOT_LEN, PIVOT_LEN)
    return df

def merge_htf_bias(ltf_df, htf_df):
    """
    که *قبل از* بسته شدنش تمام شده را پیدا میکند M15 آخرین کندل، M5 برای هر کندل
    (lookahead_off با request.security بدون نگاه به آینده - معادل).
    """
    htf_df = htf_df.set_index("timestamp").sort_index()
    merged_ema = []
    merged_close = []
    sw_high1, sw_high2, sw_low1, sw_low2 = np.nan, np.nan, np.nan, np.nan
    htf_ptr = -1
    htf_times = htf_df.index.values
    htf_records = htf_df.to_dict("records")

    bull_bias_list, bear_bias_list = [], []

    for ts in ltf_df["timestamp"]:
        # بسته شده M5 تا آخرین کندلی که کاملاً قبل از این کندل M15 پیشروی نشانگر
        while htf_ptr + 1 < len(htf_times) and htf_times[htf_ptr + 1] < np.datetime
            htf_ptr += 1
            rec = htf_records[htf_ptr]

```

```

        if rec["pivot_high"]:
            sw_high2 = sw_high1
            sw_high1 = rec["high"]
        if rec["pivot_low"]:
            sw_low2 = sw_low1
            sw_low1 = rec["low"]

    if htf_ptr < EMA_SLOPE_LOOKBACK or htf_ptr < 0:
        merged_ema.append(np.nan)
        merged_close.append(np.nan)
        bull_bias_list.append(False)
        bear_bias_list.append(False)
        continue

    cur = htf_records[htf_ptr]
    prev_ema = htf_records[htf_ptr - EMA_SLOPE_LOOKBACK]["ema"]
    htf_ema_val = cur["ema"]
    htf_close_val = cur["close"]

    bull_structure = (not np.isnan(sw_high1) and not np.isnan(sw_high2)
                      and not np.isnan(sw_low1) and not np.isnan(sw_low2)
                      and sw_high1 > sw_high2 and sw_low1 > sw_low2)
    bear_structure = (not np.isnan(sw_high1) and not np.isnan(sw_high2)
                     and not np.isnan(sw_low1) and not np.isnan(sw_low2)
                     and sw_high1 < sw_high2 and sw_low1 < sw_low2)

    price_above = htf_close_val > htf_ema_val
    price_below = htf_close_val < htf_ema_val
    slope_up = htf_ema_val > prev_ema
    slope_down = htf_ema_val < prev_ema

    bull_bias = (price_above or bull_structure) and slope_up
    bear_bias = (price_below or bear_structure) and slope_down

    merged_ema.append(htf_ema_val)
    merged_close.append(htf_close_val)
    bull_bias_list.append(bool(bull_bias))
    bear_bias_list.append(bool(bear_bias))

    ltf_df = ltf_df.copy()
    ltf_df["htf_bull_bias"] = bull_bias_list
    ltf_df["htf_bear_bias"] = bear_bias_list
    return ltf_df

def backtest(ltf_df):
    equity = INITIAL_CAPITAL

```

```

position = None # dict: side, entry, sl, tp, qty
trades = []

last_swing_low, last_swing_high = None, None
signals_today = 0
last_day = None

n = len(ltf_df)
for i in range(max(EMA_LEN, PIVOT_LEN * 2, ADX_LEN * 3), n):
    row = ltf_df.iloc[i]
    confirm_idx = i - PIVOT_LEN
    if confirm_idx >= PIVOT_LEN:
        if ltf_df["pivot_low"].iloc[confirm_idx]:
            last_swing_low = ltf_df["low"].iloc[confirm_idx]
        if ltf_df["pivot_high"].iloc[confirm_idx]:
            last_swing_high = ltf_df["high"].iloc[confirm_idx]

    day = row["timestamp"].day
    if last_day is None or day != last_day:
        signals_today = 0
        last_day = day

    high, low, close, open_ = row["high"], row["low"], row["close"], row["open"]

    # ---- مدیریت پوزیشن باز ----
    if position is not None:
        side = position["side"]
        exit_price, exit_reason = None, None
        if side == "long":
            if low <= position["sl"]:
                exit_price, exit_reason = position["sl"], "stop_loss"
            elif high >= position["tp"]:
                exit_price, exit_reason = position["tp"], "take_profit"
        else:
            if high >= position["sl"]:
                exit_price, exit_reason = position["sl"], "stop_loss"
            elif low <= position["tp"]:
                exit_price, exit_reason = position["tp"], "take_profit"

        if exit_price is not None:
            if side == "long":
                pnl = position["qty"] * (exit_price - position["entry"])
            else:
                pnl = position["qty"] * (position["entry"] - exit_price)
            fee = (position["qty"] * position["entry"] + position["qty"] * exit_price)
            pnl -= fee
            equity += pnl

```

```

        trades.append({
            "side": side, "entry_time": position["entry_time"], "entry":
            "exit_time": row["timestamp"], "exit": exit_price, "reason":
            "pnl": pnl, "equity_after": equity,
        })
        position = None

    if pd.isna(row["adx"]) or pd.isna(row["atr5"]) or pd.isna(row["ema50"]) or
        continue

# ---- بررسی سیگنال ورود جدید ----
if position is None and signals_today < MAX_SIGNALS_PER_DAY:
    dist_from_ema = abs(close - row["ema50"])
    not_too_far = dist_from_ema <= MAX_ATR_DIST * row["atr5"]
    pullback_ok = dist_from_ema <= PULLBACK_ATR_MAX * row["atr5"]
    not_sideways = row["adx"] >= ADX_MIN

    candle_range = (high - low) if (high - low) > 0 else 1e-9
    body_ratio = abs(close - open_) / candle_range
    bull_candle = close > open_ and body_ratio >= MIN_BODY_RATIO
    bear_candle = close < open_ and body_ratio >= MIN_BODY_RATIO

    buy_cond = (row["htf_bull_bias"] and close > row["ema50"] and pullbac
                and not_sideways and bull_candle and row["rsi14"] > RSI_B
    sell_cond = (row["htf_bear_bias"] and close < row["ema50"] and pullba
                and not_sideways and bear_candle and row["rsi14"] < RSI_

    if buy_cond and last_swing_low is not None and close > last_swing_low
        risk = close - last_swing_low
        risk_amount = equity * (RISK_PCT / 100)
        qty = risk_amount / risk
        position = {
            "side": "long", "entry": close, "sl": last_swing_low,
            "tp": close + risk * RR_MIN, "qty": qty, "entry_time": row["t
        }
        signals_today += 1
    elif sell_cond and last_swing_high is not None and last_swing_high >
        risk = last_swing_high - close
        risk_amount = equity * (RISK_PCT / 100)
        qty = risk_amount / risk
        position = {
            "side": "short", "entry": close, "sl": last_swing_high,
            "tp": close - risk * RR_MIN, "qty": qty, "entry_time": row["t
        }
        signals_today += 1

trades_df = pd.DataFrame(trades)

```

```

return equity, trades_df

def main():
    exchange = ccxt.binanceusdm({"enableRateLimit": True})
    print(f"برای {SYMBOL} در حال دانلود دیتای {HTF}")
    htf_df = fetch_ohlcv(exchange, SYMBOL, HTF, MONTHS_BACK)
    htf_df = add_htf_indicators(htf_df)

    print(f"برای {SYMBOL} در حال دانلود دیتای {LTF}")
    ltf_df = fetch_ohlcv(exchange, SYMBOL, LTF, MONTHS_BACK)
    ltf_df = add_ltf_indicators(ltf_df)

    print("...(بدون نگاه به آینده) M5 با کندل‌های M15 در حال هماهنگسازی بایاس")
    ltf_df = merge_htf_bias(ltf_df, htf_df)

    print("در حال اجرای بکتست")
    final_equity, trades_df = backtest(ltf_df)

    trades_df.to_csv("sync_scalper_trades.csv", index=False)

    total_trades = len(trades_df)
    wins = (trades_df["pnl"] > 0).sum() if total_trades else 0
    win_rate = (wins / total_trades * 100) if total_trades else 0
    total_pnl = trades_df["pnl"].sum() if total_trades else 0

    print("\n=== نتیجه ی بکتست Sync Scalper (M15/M5) ===")
    print(f"ماه {MONTHS_BACK} بازه: {LTF} / M5={LTF} | M15={HTF} | نماد: {SYMBOL}")
    print(f"سرمایه اولیه: ${INITIAL_CAPITAL}")
    print(f"سرمایه نهایی: ${round(final_equity, 2)}")
    print(f"سود/زیان خالص: ${round(total_pnl, 2)}")
    print(f"تعداد معاملات: {total_trades}")
    print(f"نرخ برد: {round(win_rate, 2)}%")
    print(f"\nsync_scalper_trades.csv: فایل معاملات ذخیره شد")

if __name__ == "__main__":
    main()

```